Constrained Decoding Engine — Development Guide

Oct 20, 2026 · @Kunal

Overview — what you're building and why it matters

You're building a constrained decoding engine: a library that *guarantees* an LLM's output conforms to a schema or grammar — valid JSON, a specific tool-call shape, a small DSL — by masking invalid tokens at each decode step, so the output is correct by construction rather than correct if you're lucky. You benchmark it against ordinary prompt-and-hope generation.

Why it's a strong signal for applied-AI and FDE roles:

- Reliable structured output is the single most common failure point in real LLM products — tool calling, data extraction, and agents all break on malformed output — so this solves a pain every team feels.
- It leans on a systems edge: the hard engineering is compiling a schema or grammar into a token-level state machine and masking logits every step, not prompt-wrangling.
- It's small enough to finish and rigorous enough to prove, with a clean headline number.

The headline you're aiming for: ~100% schema-valid output versus a much lower rate from prompting alone, at a modest latency overhead.

The core concept

Four ideas, in order:

1. Generation picks each next token from a probability distribution over the whole vocabulary. Nothing in that process stops the model from emitting a token that breaks your JSON — a stray quote, a wrong type, an unclosed brace.
2. The insight: at every decode step, given the grammar and what's been generated so far, you know exactly which tokens *could* legally come next. Set the logit of every illegal token to −∞ before sampling. Then whatever gets sampled keeps the output valid — validity is enforced, not hoped for.
3. To know which tokens are legal next, compile the schema or grammar into a state machine: a finite automaton for regex-like patterns, a pushdown automaton (with a stack) for nested, recursive structures like JSON. The current state tells you the

allowed next characters.

4. The hard part — tokenizer misalignment: the automaton reasons in characters, but the model emits tokens (subwords), and a single token can span several grammar transitions or straddle a structural boundary. You precompute, per automaton state, the set of allowed token IDs — an index. Building that bridge correctly and quickly is the real engineering of this project.

Scope and deliverables

MVP (must have):

- Regex / finite-automaton-constrained generation (constrain output to a date pattern, an enum, a fixed format).
- JSON-schema-constrained generation (objects, arrays, types, required fields).
- A `LogitsProcessor` that plugs into Hugging Face `generate`.
- A benchmark comparing schema-valid rate and latency against prompt-only and prompt-and-retry.

Stretch (after MVP works):

- A small EBNF context-free grammar compiled to a tool-call DSL.
- A precomputed per-state token-mask index for speed, instead of recomputing each step.
- Streaming output with early validation.

Out of scope (say so in the README):

- Any fine-tuning or training — this is pure inference-time control.
- Beating Outlines/XGrammar on raw speed.
- A full general-purpose parser for arbitrary languages.

Definition of done: given a schema, output parses as that schema 100% of the time across a varied test set, with measured latency overhead and a correctness proof against a prompt-only baseline.

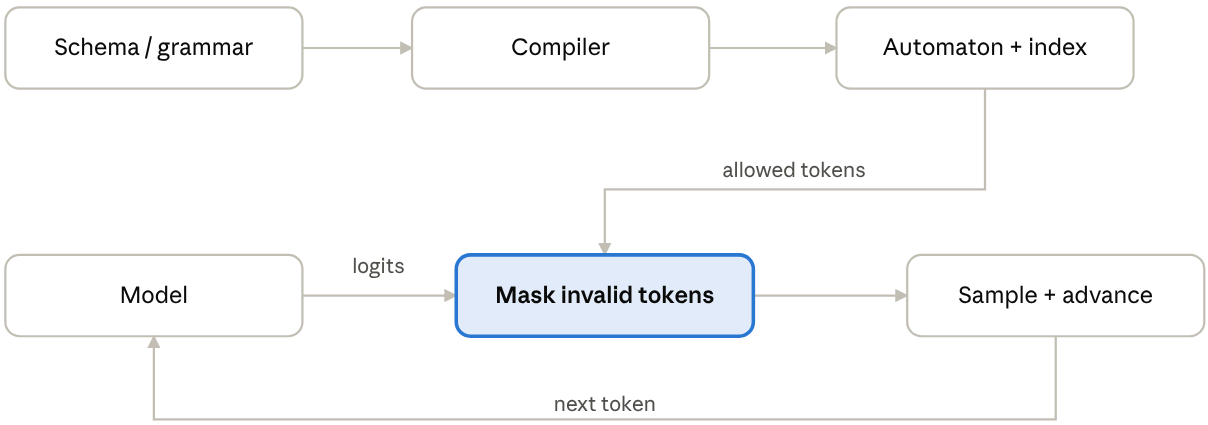
Prerequisites

- Python, PyTorch, Hugging Face `transformers`.

- A small instruct model: Qwen2.5-0.5B/1.5B-Instruct or Llama-3.2-1B-Instruct. No GPU is strictly needed for correctness (a small model runs on CPU); use a GPU only for latency numbers.
- Comfort with: regular expressions and finite automata basics, JSON Schema, how `generate` exposes a logits-processor hook, and tokenizers (vocabulary, how text maps to token IDs, subword boundaries).
- One reference to read *after* you've attempted your own: how Outlines and guidance implement this.

Architecture — how the pieces fit

Masking invalid tokens each step makes output valid by construction



compile once, then mask every decode step

The grammar is compiled once into an automaton. Then, on every decode step, the masker uses the automaton's current state to zero out illegal tokens before sampling, and advances the state with the chosen token — so the growing output can never leave the grammar.

Implementation roadmap

Each phase ends with something measurable. Build the harness before the engine so you have a baseline valid-rate to beat.

Phase	Focus	Output / gate	Rough effort
0	Baseline + harness	Prompt-only generation; measured schema-valid rate	2–3 days
1	Regex / FSA masking	Constrain to patterns; valid rate → 100%	3–5 days
2	JSON schema	Compile schema → pushdown automaton; nested objects and arrays	1–2 weeks
3	Token index + grammar (stretch)	Precomputed per-state masks; an EBNF tool-call DSL	~1 week

Phases 0–2 are the portfolio piece. Phase 3 is where the speed and generality story lives.

Phase 0 — baseline and harness (build first)

You can't claim "100% valid" without showing what prompting alone achieves. So measure that first.

1. Assemble a test set of schemas of varied complexity (a flat object, nested objects, arrays, enums, optional fields).
2. For each, prompt the model to "return JSON matching this schema," parse the raw output, and record: percentage that parses as valid JSON, percentage that also satisfies the schema, and generation latency.
3. Save results to CSV and reuse this harness unchanged for every later phase.

The low valid-rate you measure here is your problem statement, in numbers. Expect it to be clearly below 100%, especially for nested schemas and smaller models.

Core build

Five pieces; build the regex path end-to-end before touching JSON.

1. **Automaton from the grammar.** Start with regex → a finite automaton (hand-roll Thompson construction, or use a library). A state is a position in the pattern; transitions fire on characters.

2. **Char→token bridge.** At each decode step, from the current state, compute the set of token IDs whose text keeps the automaton on a valid path. The naive version: for each candidate token, check whether appending its characters leads to a live state. This is correct but slow — optimize later with a precomputed per-state index.
3. **Logit masking.** Implement a `LogitsProcessor` that sets every disallowed token's logit to −∞, then let sampling proceed normally over what remains.
4. **State advance.** After a token is chosen, advance the automaton by that token's characters so the next step masks correctly.
5. **JSON schema.** Extend from a flat automaton to a pushdown automaton — a stack tracks nested braces and brackets — and map the schema (types, required keys, enums) onto grammar rules.

Milestone order: regex FSA working end-to-end through the decode loop first (prove the masking loop is correct), then JSON schema, then speed. Don't skip the regex step — it's where you debug the core loop on something simple.

Validation and metrics

Correctness is the headline, so prove it. Across the test set, 100% of outputs must parse as the target schema. Any failure is an automaton bug, not an acceptable miss.

Edge cases to test explicitly: empty objects, nested arrays, escaped characters in strings, unicode, numbers vs. strings, optional fields, and the nasty one — a single token that straddles a structural boundary (e.g. a token that is `",`).

Metrics to report:

- Schema-valid rate: constrained (should be ~100%) vs. prompt-only vs. prompt-and-retry.
- Latency overhead per token from masking, and tokens/sec vs. the unconstrained baseline.
- Retries eliminated: how many prompt-and-retry attempts your approach saves on average.

Headline sentence: "100% schema-valid output vs. ~X% from prompting, at ~Y% latency overhead and zero retries."

Repo, README, and interview points

Repo layout: `engine/` (automaton, masker, the `LogitsProcessor`), `grammars/`, `bench/`, `tests/`, `results/`, `README.md`.

README order: one-line what + the valid-rate headline up top; the problem (why prompting fails for structured output); the architecture (link the diagram above); the metrics table; one command to run; scope in/out.

Interview framing:

- Open with the pain: "Structured output is where most LLM products break, and prompting can't guarantee it."
- Name the non-obvious hard part: the tokenizer-boundary problem, where the automaton thinks in characters and the model thinks in tokens.
- Lead with numbers: 100% validity, latency trade-off, retries removed.
- Have a "what broke" story: a token straddling a brace, and the per-state index that fixed both correctness and speed.
- Connect to the role: every tool-calling and agent system needs exactly this guarantee.

Gotchas and resources

Gotchas:

- Tokenizer-boundary bugs are the whole difficulty — budget most of your debugging time here.
- Recomputing the allowed-token set every step is slow; precompute a per-state index once the logic is correct.
- Masking *every* token to −∞ deadlocks generation — always leave at least one legal token, and handle the end-of-sequence token so the model can actually stop.
- Over-constraining so the structure can never legally close (unbalanced braces) — test termination explicitly.
- Testing only happy-path schemas; the value is in the awkward ones.

Resources:

- Outlines (and its finite-automaton approach to constrained generation) — the clearest reference.
- guidance, and XGrammar / llguidance for fast per-state masking ideas.
- Hugging Face `LogitsProcessor` docs.

- A primer on regex-to-automaton compilation (Thompson construction).

This area moves fast, so check the current state of the art (XGrammar and newer grammar engines) before locking your design. I can pull the latest if you want.